

Aula 04 - Envio de Dados e Ciclo CRUD no Front-End

Descrição: Os estudantes desenvolvem formulários interativos na interface web para enviar e manipular dados em uma API RESTful. A prática contempla a interceptação de eventos, a serialização de dados em formato JSON e a execução de requisições POST com validação do status HTTP 201, além da implementação dos métodos PUT/PATCH e DELETE para alteração e exclusão de registros, concluindo o ciclo CRUD no Front-End com a sincronização automática do estado visual da tela.

O que vamos construir hoje?

Na aula passada, criamos um painel de logística que **apenas lê** as encomendas de um arquivo. Ele funciona como uma foto ou um quadro de avisos de cortiça: você consegue ver os papéis colados, mas não consegue adicionar um pacote novo, mudar a entrega para "Entregue" ou rasgar uma encomenda cancelada.

Hoje, vamos transformar essa tela estática em um **sistema interativo e moderno (SPA — Single Page Application)**!

Ao final desta aula, você terá um painel completo capaz de:

1. **Cadastrar novas encomendas** sem a tela piscar ou dar refresh (adeus, tela branca!).
2. **Alterar o status** da entrega na hora com um simples clique direto na tabela.
3. **Excluir encomendas** canceladas com um botão dedicado.
4. **Salvar tudo na memória do navegador** (`localStorage`), para que nada se perca mesmo se você fechar e abrir o navegador de novo!

Etapa 1: Ponto de Partida e o Problema do "F5"

Abra a pasta do projeto no VS Code e inicie o **Live Server** clicando em **Go Live** (no cantinho inferior direito da barra de status).

O "Teste do Formulário Fantasma"

Insira um formulário simples no final do nosso `index.html`:

HTML

```
<form id="form-teste">
  <input type="text" placeholder="Código de Rastreo" required>
  <button type="submit">Cadastrar Encomenda</button>
</form>
```

1. Digite qualquer código de rastreo no campo.
2. Clique no botão **"Cadastrar Encomenda"**.
3. **Observe o que aconteceu:** A página inteira piscou, deu um "F5" forçado, a URL lá em cima mudou e o que você digitou desapareceu!

Pense rápido:

Quando você envia uma mensagem no Discord ou WhatsApp Web, a página inteira recarrega e fica branca por um segundo? Não! O aplicativo manda a mensagem em segundo plano e coloca o balãozinho na tela na mesma hora. É exatamente essa experiência de usuário (UX) profissional que vamos construir hoje.

Etapa 2: Nossa "Central de Dados" no Navegador (**localStorage**)

Em um sistema empresarial, os dados vão pela internet até um servidor com banco de dados. Mas por que não podemos simplesmente gravar dados direto no arquivo **encomendas.json** pelo JavaScript?

Regra de Ouro da Web: Por segurança, o navegador **não tem permissão** para abrir o seu computador e alterar arquivos locais do seu disco.

Para resolver isso sem precisar instalar nenhum programa bloqueado pelo laboratório, vamos usar o **localStorage**: uma memória interna e permanente que todo navegador moderno possui para guardar informações no formato de texto JSON.

Passo 2.1 - Criando a Simulação de Rede no **app.js**

Na internet real, os dados levam alguns milissegundos para viajar. Para aprendermos a lidar com o tempo de espera e dar avisos na tela para o usuário, vamos criar uma pequena função de "atraso" (*delay*).

Abra o seu **app.js** e adicione logo no topo (linha 1):

JavaScript

```
// Simula o tempo que a internet levaria para responder (em milissegundos)
const simularRede = (ms = 300) => new Promise(resolve => setTimeout(resolve, ms));
```

Passo 2.2 - Atualizando a Consulta de Encomendas

Vamos ensinar a nossa função **consultarEncomendas** a fazer o seguinte:

1. Se for a **primeira vez** que você abre a página, ela busca as encomendas do **encomendas.json** (como já fazia) e guarda uma cópia no **localStorage**.
2. Nas vezes seguintes, ela passa a ler direto da memória do **localStorage**.

No **app.js**, substitua a função **consultarEncomendas** por esta versão atualizada:

JavaScript

```
async function consultarEncomendas() {
  elCarregando.style.display = "block";
  elErro.style.display = "none";

  try {
    // Simula uma resposta rápida de rede
    await simularRede(300);

    // 1. Tenta pegar os dados salvos na memória do navegador
    let dadosSalvos = localStorage.getItem("banco_encomendas");

    // 2. Se a memória estiver vazia, busca do encomendas.json pela primeira vez
    if (!dadosSalvos) {
      const resposta = await fetch("encomendas.json");
      if (!resposta.ok) throw new Error("Erro ao ler o arquivo JSON");

      const dadosIniciais = await resposta.json();

      // Guarda no navegador em formato de texto JSON
    }
  }
}
```

```
localStorage.setItem("banco_encomendas", JSON.stringify(dadosIniciais));
dadosSalvos = JSON.stringify(dadosIniciais);
}

// 3. Converte o texto JSON de volta para uma lista de objetos no JavaScript
const encomendas = JSON.parse(dadosSalvos);
renderizarTabela(encomendas);

} catch (erro) {
  elErro.textContent = "Não foi possível carregar as encomendas.";
  elErro.style.display = "block";
  console.error(erro);
} finally {
  elCarregando.style.display = "none";
}
}
```

Conceito Chave:

- **JSON.stringify(objeto):** Transforma dados do JavaScript em texto puro para poder salvar.
- **JSON.parse(texto):** Transforma o texto de volta em dados que o JavaScript entende.

Etapa 3: Formulários Sem F5 e Cadastro (Create)

Passo 3.1 — Ajustando o `index.html`

Vamos preparar nossa interface:

1. Adicionar a coluna **"Ações"** no cabeçalho da tabela.
2. Trocar o formulário de teste por um formulário completo com código, nome e seleção de status.

Abra o `index.html` e substitua todo o seu conteúdo por:

```
HTML
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Painel de Encomendas</title>
</head>
<body>
  <h1>Painel de Controle de Entregas</h1>

  <p id="carregando">Carregando encomendas...</p>
  <p id="erro" style="color: red; display: none;"></p>

  <!-- Formulário Completo de Cadastro -->
  <form id="form-encomenda" style="margin-bottom: 25px;">
    <h3>Cadastrar Nova Encomenda</h3>

    <input type="text" id="input-codigo" placeholder="Código (ex: BR004)" required>
```

```
<input type="text" id="input-cliente" placeholder="Nome do Cliente" required>

<select id="select-status">
  <option value="PENDENTE">PENDENTE</option>
  <option value="EM_TRANSITO">EM TRÂNSITO</option>
  <option value="ENTREGUE">ENTREGUE</option>
</select>

<button type="submit">Cadastrar Encomenda</button>
</form>

<table id="tabela-encomendas" border="1" cellpadding="6" style="border-collapse: collapse;">
  <thead>
    <tr>
      <th>Código</th>
      <th>Cliente</th>
      <th>Status</th>
      <th>Ações</th> <!-- Nova coluna para os botões! -->
    </tr>
  </thead>
  <tbody id="corpo-tabela"></tbody>
</table>

<script src="app.js"></script>
</body>
</html>
```

Passo 3.2 — Puxando o "Freio de Mão" (`event.preventDefault()`)

Para impedir que o navegador recarregue a página quando o formulário for enviado, usamos o comando `event.preventDefault()`.

No final do `app.js`, adicione o ouvinte de envio:

JavaScript

```
// Seleciona o formulário oficial
const formEncomenda = document.getElementById("form-encomenda");

// Escuta o momento exato em que o usuário clica no botão ou aperta Enter
formEncomenda.addEventListener("submit", async function(event) {
  // PUXA O FREIO DE MÃO: Impede a tela de piscar/recarregar!
  event.preventDefault();

  // Captura os valores digitados
  const novaEncomenda = {
    codigoRastreio: document.getElementById("input-codigo").value.trim(),
    clienteNome: document.getElementById("input-cliente").value.trim(),
    status: document.getElementById("select-status").value
  };

  // Envia para a função de salvamento
```

```
    await cadastrarEncomenda(novaEncomenda);  
  });
```

Passo 3.3 — Criando a Função `cadastrarEncomenda`

Quando o usuário clica em cadastrar, queremos uma boa experiência de uso (UX): desabilitar o botão temporariamente para evitar cliques duplos e avisar que o sistema está processando.

Adicione a função no seu `app.js`:

JavaScript

```
async function cadastrarEncomenda(novaEncomenda) {  
  const btnSubmit = document.querySelector("#form-encomenda button[type='submit']");  
  
  try {  
    // Feedback visual imediato: trava o botão enquanto salva  
    btnSubmit.disabled = true;  
    btnSubmit.textContent = "Cadastrando...";  
  
    // Simula a espera do processamento (500ms)  
    await simularRede(500);  
  
    // 1. Pega as encomendas que já estão guardadas  
    const encomendas = JSON.parse(localStorage.getItem("banco_encomendas")) || [];  
  
    // 2. Adiciona o novo item na lista  
    encomendas.push(novaEncomenda);  
  
    // 3. Salva a lista atualizada de volta na memória  
    localStorage.setItem("banco_encomendas", JSON.stringify(encomendas));  
  
    // 4. A REGRA DE OURO (Fonte da Verdade): Chama a consulta para redesenhar a tela  
    await consultarEncomendas();  
  
    // 5. Limpa os campos do formulário  
    document.getElementById("form-encomenda").reset();  
  
  } catch (erro) {  
    console.error("Erro ao salvar:", erro);  
    alert("Ops! Ocorreu um erro ao salvar a encomenda.");  
  
  } finally {  
    // Restaura o botão ao normal, dando certo ou errado  
    btnSubmit.disabled = false;  
    btnSubmit.textContent = "Cadastrar Encomenda";  
  }  
}
```

Faça o teste agora: Cadastre uma encomenda no formulário. Repare que o botão muda de texto rapidamente e a nova linha entra na tabela sem que a página recarregue!

Etapa 4: Atualizando (Update) e Excluindo (Delete)

Agora que sabemos criar novos itens, precisamos conseguir **modificar** o status e **remover** pacotes cancelados.

Passo 4.1 - Criando as Funções de Alteração e Exclusão

Adicione estas duas funções no `app.js`:

JavaScript

// EXCLUIR ENCOMENDA

```
async function excluirEncomenda(codigo) {
  try {
    await simularRede(200);

    // 1. Pega a lista atual
    let encomendas = JSON.parse(localStorage.getItem("banco_encomendas")) || [];

    // 2. Filtra a lista, mantendo apenas quem NÃO TEM esse código
    encomendas = encomendas.filter(item => item.codigoRastreio !== codigo);

    // 3. Salva a nova lista sem o item deletado
    localStorage.setItem("banco_encomendas", JSON.stringify(encomendas));

    // 4. Redesenha a tabela
    await consultarEncomendas();

  } catch (erro) {
    console.error("Erro ao excluir:", erro);
    alert("Não foi possível excluir a encomenda.");
  }
}
```

// ATUALIZAR STATUS DA ENCOMENDA

```
async function atualizarStatus(codigo, novoStatus) {
  try {
    await simularRede(200);

    const encomendas = JSON.parse(localStorage.getItem("banco_encomendas")) || [];

    // Localiza o item pelo código de rastreio
    const itemEncontrado = encomendas.find(item => item.codigoRastreio === codigo);

    if (itemEncontrado) {
      itemEncontrado.status = novoStatus; // Altera o status
      localStorage.setItem("banco_encomendas", JSON.stringify(encomendas));
      await consultarEncomendas();
    }

  } catch (erro) {
```

```
    console.error("Erro ao atualizar status:", erro);  
    alert("Falha ao atualizar o status.");  
  }  
}
```

Passo 4.2 - Atualizando a `renderizarTabela` com Controles Interativos

Agora vamos redesenhar a tabela para que cada linha ganhe seu próprio seletor de status (`<select>`) e um botão de exclusão (🗑 `Excluir`).

No seu `app.js`, substitua a função `renderizarTabela` existente por esta:

JavaScript

```
function renderizarTabela(encomendas) {  
  corpoTabela.innerHTML = ""; // Limpa a tabela antes de redesenhar  
  
  const fragmento = document.createDocumentFragment();  
  
  encomendas.forEach(encomenda => {  
    const linha = document.createElement("tr");  
  
    // 1. Coluna Código  
    const tdCodigo = document.createElement("td");  
    tdCodigo.textContent = encomenda.codigoRastreio;  
  
    // 2. Coluna Cliente  
    const tdCliente = document.createElement("td");  
    tdCliente.textContent = encomenda.clienteNome;  
  
    // 3. Coluna Status com Seletor Interativo  
    const tdStatus = document.createElement("td");  
    const selectStatus = document.createElement("select");  
  
    const opcoes = ["PENDENTE", "EM_TRANSITO", "ENTREGUE"];  
    opcoes.forEach(opcao => {  
      const opt = document.createElement("option");  
      opt.value = opcao;  
      opt.textContent = opcao.replace("_", " ");  
      if (opcao === encomenda.status) opt.selected = true;  
      selectStatus.appendChild(opt);  
    });  
  
    // Quando o usuário escolhe outra opção, atualiza automaticamente!  
    selectStatus.addEventListener("change", (e) => {  
      atualizarStatus(encomenda.codigoRastreio, e.target.value);  
    });  
  
    tdStatus.appendChild(selectStatus);  
  
    // 4. Coluna Ações com Botão de Excluir  
    const tdAcoes = document.createElement("td");
```

```
const btnExcluir = document.createElement("button");
btnExcluir.textContent = "🗑 Excluir";

btnExcluir.addEventListener("click", () => {
  excluirEncomenda(encomenda.codigoRastreio);
});

tdAcoes.appendChild(btnExcluir);

// Monta as 4 colunas na linha
linha.appendChild(tdCodigo);
linha.appendChild(tdCliente);
linha.appendChild(tdStatus);
linha.appendChild(tdAcoes);

fragmento.appendChild(linha);
});

corpoTabela.appendChild(fragmento);
}
```

Etapa 5: O Laboratório Dev no Navegador

Vamos inspecionar a memória do navegador para ver onde os dados realmente estão morando!

1. Na página aberta pelo Live Server, aperte a tecla **F12** (ou clique com o botão direito e vá em **Inspecionar**).
2. No menu superior da janela do desenvolvedor, clique na aba **Application** (ou **Aplicativo**).
3. No menu lateral esquerdo, clique na setinha ao lado de **Local Storage** e escolha o endereço do seu Live Server (`[http://127.0.0.1:5500](http://127.0.0.1:5500)`).
4. Veja a chave `banco_encomendas`:
 - Cadastre um novo pacote e veja o texto aumentar em tempo real!
 - Mude o status ou clique em excluir e veja a linha no DevTools se atualizar sozinha.
 - **O teste supremo:** Dê um **F5** na página ou feche o navegador e abra de novo. Seus dados continuam todos lá intactos!

Etapa 6: Missão Prática em Duplas

Trabalhem em duplas e confirmem o checklist de desenvolvimento do painel:

- [] **Sem F5:** O formulário cadastra novos itens sem piscar a página.
- [] **Feedback:** O botão exibe "**Cadastrando...**" e fica desabilitado durante a gravação.
- [] **Update Direto:** Ao mudar o status no seletor da tabela, a alteração se mantém após recarregar a página.
- [] **Delete:** Clicar em "🗑 Excluir" remove a linha da tabela e apaga do `localStorage`.
- [] **Persistência:** Fechar a aba do navegador e abrir novamente mantém todos os dados cadastrados.

Mapa Mental da Aula (Card de Consulta Rápida)

Recurso	Para que serve no projeto?
<code>event.preventDefault()</code>	Puxa o "freio de mão" do formulário para impedir que a tela recarregue (F5).
<code>localStorage.setItem()</code>	Salva dados de forma permanente na memória do navegador.

<code>localStorage.getItem()</code>	Recupera os dados salvos anteriormente no navegador.
<code>JSON.stringify()</code>	Converte objetos e listas JavaScript em texto para salvar no armazenamento.
<code>JSON.parse()</code>	Converte o texto salvo de volta para objetos/listas que o JavaScript manipula.
Fonte Única da Verdade	Sempre que alteramos dados, chamamos <code>consultarEncomendas()</code> para manter a tela sincronizada com o banco.

Spoiler das Próximas Aulas:

Hoje aprendemos a dominar a tela e os dados do lado do usuário (**Front-End**)! Nas próximas aulas, vamos aprender como construir o outro lado da ponte: o **Back-End com Node.js**, criando APIs que recebem esses mesmos dados e gravam em um banco de dados real no servidor!